

#03 Time Complexity & Basic Data Structure

김성수

4thrunyul@gmail.com

0. 목차



1. 아이스 브레이킹

- 동전 수 같게 하기

2. 문제풀이 전략

- 피드백

3. 시간 복잡도

- 빅오(Big-O) 표기법

4. 기초 자료구조

- vector
- pair

5. 잡다하지만 중요한 내용

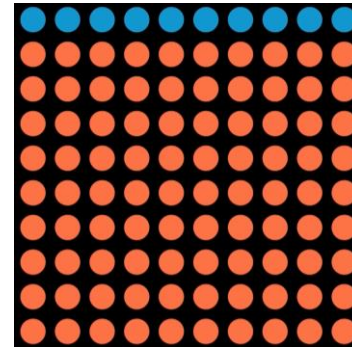
1. 아이스 브레이킹: 동전 수 같게 하기

전제

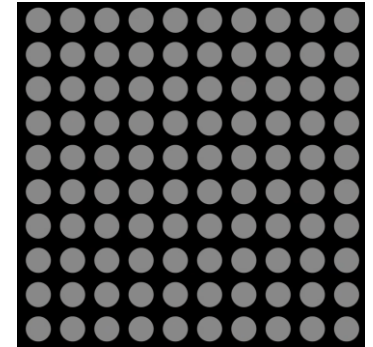
- 100개의 동전이 있음.
- 이 동전들의 앞면은 **파란색**, 뒷면은 **빨간색**임.
- 현재 앞면인 동전은 10개, 뒷면인 동전은 90개임.

문제

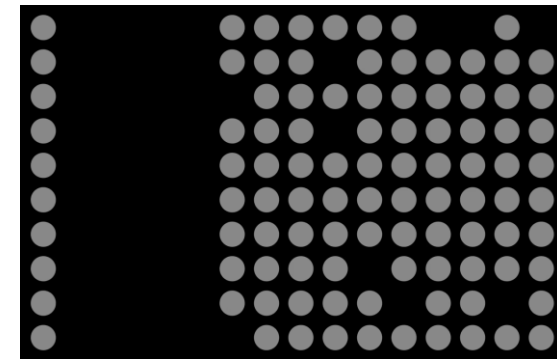
- 선글라스를 꺼서 동전의 색이 구분되지 않음.
- 100개의 동전들 중에서 무작위로 10개의 동전을 골라 집합 A로 만들고, 남은 90개는 집합 B로 만듦.
- 이때 집합 A와 집합 B의 **파란색** 동전 수를 같게 하는 방법은?



▲ 원래 상태



▲ 선글라스를 낀 상태



▲ 집합 A와 B

1. 아이스 브레이킹: 동전 수 같게 하기

풀이

- 집합 A의 파란색 동전 개수를 X 개라고 하면, 집합 B에는 $10 - X$ 개 만큼의 파란색 동전이 남아 있음.
- 한편, 집합 A의 빨간색 동전 개수도 $10 - X$ 개.

정답

- 집합 A의 모든 동전 뒤집기!

예시

- 집합 A에 파란색 동전 2개, 집합 B에 파란색 동전 8개가 있음.
- 집합 A에 빨간색 동전 8개가 있음.
- 집합 A에 있는 모든 동전을 뒤집으면 최종적으로 파란색 동전 8개, 빨간색 동전 2개로 남음.

2. 문제 풀이 전략: 순서



문제 파악 - 수도 코드 작성 - 구현 - 디버깅 - 제출 - **피드백**

* 굵은 표시가 되어있는 부분은 이번 시간에 주로 다룰 것을 표시한 것이지, 중요도를 나타낸 게 아님

2. 문제 풀이 전략: 피드백



피드백, 언제 해야 하는가?

- 일정 횟수 이상 코드를 제출했음에도 문제가 풀리지 않을 때
- **맞았습니다!!**를 받았음에도 불구하고 더 효율적인 방법이 있었을 것 같을 때

피드백, 어떻게 해야 하는가?

- AI 사용하기
- TIL(Today I Learn)에 정리하기

2. 문제 풀이 전략: 피드백



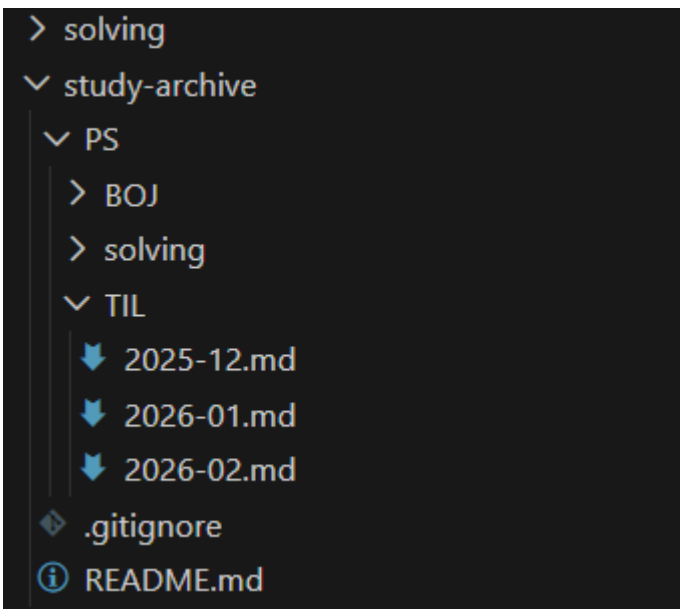
1. AI 사용하기

- AI(ChatGPT, Gemini, Claude)에 구현한 코드를 제출하면서 ‘내가 BOJ N번 문제를 풀고 있는데 이것보다 더 효율적인 로직이 있을까?’ 형식으로 물어보기

2. 문제 풀이 전략: 피드백

2. TIL(Today I Learn)에 정리하기

- TIL 폴더에 연도-월.md를 생성
- 해당 문서에는 피드백 하려는 문제의 접근 방식, 해결 방법, 배운 것에 대해 작성한다.
- 글을 정리하는 방법이 꼭 해당 방식이 아니어도 됨.



```
1 # [2025-12-24] | BOJ / 2744 - 대소문자 바꾸기 | B5
2 ## 1. Approach
3 문자열을 순회하며 ASCII 값 차이를 이용해 대소문자를 변환하고자 하였다.
4
5 ## 2. Solution
6 ```cpp
7 for(char c : s) {
8     if(c >= 'A' && c <= 'Z') cout << (char)(c + 32);
9     else cout << (char)(c - 32);
10 }
11 // #include <cctype>
12 // == (char)tolower(c), (char)toupper(c)
13 ```
14
15 ## 3. Lesson
16 - 내가 간과한 사실은 대소문자의 ASCII 값 차이가 26이 아니라 32였다는 것이다. 대문자가 65~90의 값을 가지고 있는 한편, 소문자는 97~122의 값을 가지고 있었고 그 사이에는 [, \, ], ^, _ , `에 해당하는 값들이 존재했다.
17 - `(char)c + 32`와 `(char)(c + 32)`는 다른 값을 출력한다. 첫 번째 식은 (char)가 c값 만 형변환 하는 반면, 두번째 식은 c + 32의 결과값을 문자로 형변환한다.
18 - ASCII 값의 차를 이용하지 않고 `<cctype>` 라이브러리의 변환 함수 `tolower()`, `toupper()`와 판별 함수 `islower()`, `isupper()`를 사용할 수도 있다. 참고로 판별 함수는 bool값이 아니라 int값의 1(참) 또는 0(거짓)을 반환한다.
```


2. 문제 풀이 전략: 피드백



```
1 # [2025-12-24] | BOJ / 2744 - 대소문자 바꾸기 | B5
2 ## 1. Approach
3 문자열을 순회하며 ASCII 값 차이를 이용해 대소문자를 변환하고자 하였다.
4
5 ## 2. Solution
6 ```cpp
7 for(char c : s) {
8     if(c >= 'A' && c <= 'Z') cout << (char)(c + 32);
9     else cout << (char)(c - 32);
10 }
11 // #include <cctype>
12 // == (char)tolower(c), (char)toupper(c)
13 ```
14
15 ## 3. Lesson
16 - 내가 간과한 사실은 대소문자의 ASCII 값 차이가 26이 아니라 32였다는 것이다. 대문자가 65~90의 값을 가지고 있는 한편, 소문자는 97~122의 값을 가지고 있었고 그 사이에는 [, \, ], ^, _, `에 해당하는 값들이 존재했다.
17 - `(char)c + 32`와 `(char)(c + 32)`는 다른 값을 출력한다. 첫 번째 식은 (char)가 c값 만 형변환 하는 반면, 두번째 식은 c + 32의 결과값을 문자로 형변환한다.
18 - ASCII 값의 차를 이용하지 않고 `<cctype>` 라이브러리의 변환 함수 `tolower()`, `toupper()`와 판별 함수 `islower()`, `isupper()`를 사용할 수도 있다. 참고로 판별 함수는 bool값이 아니라 int값의 1(참) 또는 0(거짓)을 반환한다.
```

2. 문제 풀이 전략: 피드백



FourthRun docs: TIL 2025-12-26

52436cd · 2 months ago History

Preview

Code

Blame

78 lines (60 loc) · 3.18 KB

Raw



[2025-12-24] | BOJ / 2744 - 대소문자 바꾸기 | B5

1. Approach

문자열을 순회하며 ASCII 값 차이를 이용해 대소문자를 변환하고자 하였다.

2. Solution

```
for(char c : s) {  
    if(c >= 'A' && c <= 'Z') cout << (char)(c + 32);  
    else cout << (char)(c - 32);  
}  
// #include <cctype>  
// == (char)tolower(c), (char)toupper(c)
```



3. Lesson

- 내가 간과한 사실은 대소문자의 ASCII 값 차이가 26이 아니라 32였다는 것이다. 대문자가 65~90의 값을 가지고 있는 한편, 소문자는 97~122의 값을 가지고 있었고 그 사이에는 [, \, ^, _ `에 해당하는 값들이 존재했다.
- `(char)c + 32` 와 `(char)(c + 32)` 는 다른 값을 출력한다. 첫 번째 식은 `(char)`가 `c`값 만 형변환 하는 반면, 두번째 식은 `c + 32`의 결과값을 문자로 형 변환한다.
- ASCII 값의 차를 이용하지 않고 `<cctype>` 라이브러리의 변환 함수 `tolower()`, `toupper()` 와 판별 함수 `islower()`, `isupper()` 를 사용할 수도 있다. 참고로 판별 함수는 bool값이 아니라 int값의 1(참) 또는 0(거짓)을 반환한다.

3. 빅오(Big-O) 표기법

프로그래머가 고려해야 할 경우?

- 최선의 경우(best case): 수행 시간이 가장 빠른 경우
- $n = 0$ 일 때, for문이 1번 만 실행되고 끝남.
- 평균의 경우(average case): 수행시간이 평균적인 경우
- $n = 4$ 혹은 $n = 5$ 일 때, for문이 4, 5번 만 실행되고 끝남.
- 최악의 경우(worst case): 수행 시간이 가장 늦은 경우
- $n = 9$ 일 때, for문이 9번 실행되고 끝남.

결론 : 최악의 경우의 수를 고려해야 함.

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int arr[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
7      int n; // 단,  $0 \leq n \leq 9$ 
8
9      cin >> n;
10
11     for(int i = 0; i < 10; i++) {
12         if(arr[i] == n) {
13             cout << "Find Number";
14
15             break;
16         }
17     }
18
19     return 0;
20 }
```

3. 빅오(Big-O) 표기법

코드 A

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, count;
7
8      cin >> n;
9
10     count = n * n;
11
12     cout << count;
13
14     return 0;
15 }
```

코드 B

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      int count = 0;
8
9      cin >> n;
10
11     for(int i = 0; i < n; i++) {
12         count += n;
13     }
14
15     cout << count;
16
17     return 0;
18 }
```

코드 C

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      int count = 0;
8
9      cin >> n;
10
11     for(int i = 0; i < n; i++) {
12         for(int j = 0; j < n; j++) {
13             count++;
14         }
15     }
16
17     cout << count;
18
19     return 0;
20 }
```

양의 정수 n 을 입력 받고 n 의 제곱을 출력하기

- 해당 코드들에는 무슨 차이가 있을까?

Input:	Copy
5	
Expected Output:	Copy
25	

3. 빅오(Big-O) 표기법

코드 A : 1번 연산

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, count;
7
8      cin >> n;
9
10     count = n * n;
11
12     cout << count;
13
14     return 0;
15 }
```

코드 B : n번 연산

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      int count = 0;
8
9      cin >> n;
10
11     for(int i = 0; i < n; i++) {
12         count += n;
13     }
14
15     cout << count;
16
17     return 0;
18 }
```

코드 C : n^2 번 연산

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      int count = 0;
8
9      cin >> n;
10
11     for(int i = 0; i < n; i++) {
12         for(int j = 0; j < n; j++) {
13             count++;
14         }
15     }
16
17     cout << count;
18
19     return 0;
20 }
```

만약 $n = 100,000,000$ (1억)이라면?

3. 빅오(Big-O) 표기법: 시간 복잡도

시간 복잡도

- 입력의 크기 N 이 커질 때 알고리즘이 수행해야 하는 연산 횟수가 어떻게 증가하는지를 나타내는 척도.
- 컴퓨터 성능, 언어, 구현 디테일은 무시하고 입력이 커졌을 때 얼마나 많이 반복하는지만 고려함.
- 대표적인 방법 중 하나가 빅오 표기법이고 그 외 빅세타, 빅오메가 등이 있다.

빅오 표기법

- 연산의 횟수를 대략적(점근적)으로 표기한 것
- 최악의 경우를 기준으로 이 이상은 절대 넘지 않는다는 상한선

3. 빅오(Big-O) 표기법: 계산 규칙

1. 상수 무시

- $f(n) = 2n$ 일 경우 $O(n)$
- $g(n) = \frac{1}{2} * n^2$ 일 경우 $O(n^2)$
- $h(n) = 9$ 일 경우 $O(1)$

2. 큰 항만 남김

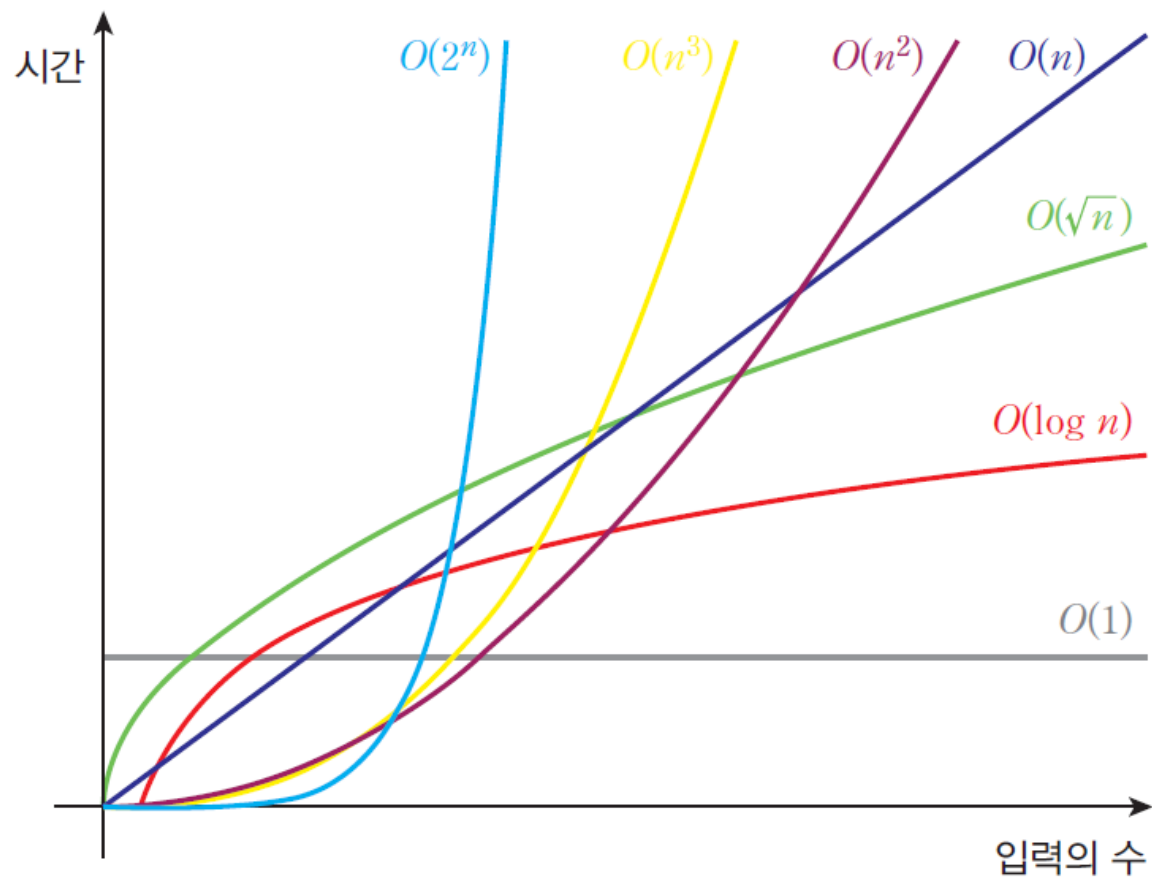
- $f(n) = 5n + 1$ 일 경우 $O(n)$
- $g(n) = 61n^2 + 2n + 152$ 일 경우 $O(n^2)$

3. 빅오(Big-O) 표기법: 계산 규칙



빅오 표기법	예시	예시값 빅오 표현	예제
$O(1)$: 상수형	10	$O(1)$	단순 연산(+, -, * 등), stack에서 pop과 push
$O(\log n)$: 로그형	$\log 4n$	$O(\log n)$	이진트리
$O(n)$: 선형	$4n$	$O(n)$	for문
$O(n \log n)$: 선형로그형	$2n \log 5n$	$O(n \log n)$	퀵 정렬, 병합 정렬, 힙 정렬
$O(n^2)$: 2차형	$5n^2 + 3n + 1$	$O(n^2)$	이중 for문, 삽입 정렬, 버블 정렬, 선택 정렬
$O(n^3)$: 3차형	$3n^3 + 5n$	$O(n^3)$	
$O(2^n)$: 지수형	2^{3n}	$O(2^n)$	피보나치 수열
$O(n!)$: 팩토리얼형	$3n!$	$O(n!)$	

3. 빅오(Big-O) 표기법



3. 빅오(Big-O) 표기법

코드 A : $O(1)$

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n, count;
7
8      cin >> n;
9
10     count = n * n;
11
12     cout << count;
13
14     return 0;
15 }
```

코드 B : $O(n)$

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      int count = 0;
8
9      cin >> n;
10
11     for(int i = 0; i < n; i++) {
12         count += n;
13     }
14
15     cout << count;
16
17     return 0;
18 }
```

코드 C : $O(n^2)$

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int n;
7      int count = 0;
8
9      cin >> n;
10
11     for(int i = 0; i < n; i++) {
12         for(int j = 0; j < n; j++) {
13             count++;
14         }
15     }
16
17     cout << count;
18
19     return 0;
20 }
```

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <Vector>
3
4  using namespace std;
5
6  int main() {
7      vector<int> v1;
8
9      vector<int> v2(3);
10
11     vector<int> v3(3, 5);
12
13     vector<int> v4 = {1, 2, 3};
14
15     return 0;
16 }
```

벡터(vector) | <vector>

- 같은 자료형의 변수들을 메모리 상에 연속적으로 나열한 자료구조
- 모든 자료형(int, float, char, bool, string 등)에서 선언 가능
- 데이터가 빈틈없이 붙어 있어 인덱스(번호)만 알면 즉시 찾을 수 있음.
- 가변 크기 : 데이터가 꽉 차면 알아서 메모리 공간을 확장 함.

선언과 초기화

- v1 : int형 벡터를 선언하였고, 현재 칸의 개수는 0임.
- v2 : int형 벡터를 3칸 선언하고, 모든 칸의 값을 0으로 초기화 함.
- v3 : int형 벡터를 3칸 선언하고, 모든 칸의 값을 5로 초기화 함.
- v4 : int형 벡터를 선언하고, 원하는 원소들로 초기화 함.

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <Vector>
3
4  using namespace std;
5
6  int main() {
7      vector<int> v = {10, 20, 30};
8
9      cout << v[0] << " ";
10
11     v[1] = 15;
12     cout << v[1] << " ";
13
14     v[2]++;
15     cout << v[2] + v[0];
16
17     return 0;
18 }
```

Input:

Copy

Expected Output:

Copy

10 15 41

벡터 인덱싱 | $O(1)$

- 배열의 첫 번째 요소는 항상 0번에서 시작함.
- 인덱싱을 통해 원소에 접근할 수 있으며, 수정하거나 출력할 수 있음.
- 반복문의 제어 변수(주로 i)를 통해 배열의 모든 원소를 탐색할 수 있음

```
1  #include <iostream>
2  #include <Vector>
3
4  using namespace std;
5
6  int main() {
7      vector<int> v = {10, 20, 30, 40, 50};
8
9      for(int i = 0; i < v.size(); i++) {
10         cout << v[i] << " ";
11     }
12
13     return 0;
14 }
```

Input:

Copy

Expected Output:

Copy

10 20 30 40 50

4. 기초 자료구조: vector



반복자(iterator)

- STL 컨테이너(객체 자료구조)에 저장된 원소들을 하나하나 순회하면서 접근할 수 있는 객체
- 배열의 포인터와 비슷한 역할로, 각 원소의 메모리 주소를 가리킴

v.begin()

- 시작점, 벡터의 첫 번째 원소를 가리키는 반복자 반환

v.end()

- 끝점, 벡터의 마지막 원소 다음 칸을 가리키는 반복자 반환
- 만약 벡터의 크기가 n이라면 `vector.begin() + n`는 `vector.end()`와 같음

```
vector<int> v = {10, 20, 30, 40, 50};
```

- 해당 구조에서 `v.begin() + 5`는 `v.end()`와 같음

10	20	30	40	50
----	----	----	----	----

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  int main() {
7      vector<int> v = {10, 20, 30, 40, 50};
8
9      cout << v.size() << "\n";
10
11     v.clear();
12
13     if(v.empty()) {
14         cout << "now vector is empty";
15     }
16
17     return 0;
18 }
```

Input:

Copy

Expected Output:

Copy

```
5
now vector is empty
```

v.size() | <vector>

- 벡터 크기 반환
- 반환해주는 값은 일반적인 int가 아니라 unsigned int임.
- unsigned int는 부호가 없는 정수로 자연수만 표현할 수 있는 값
- unsigned int로 선언된 0에 -1을 연산하면 오버플로우로 인해 4,294,967,295이 됨.

v.clear() | <vector>

- 벡터의 모든 원소 제거
- 사용한 메모리 공간 자체는 남음

v.empty() | <vector>

- 벡터가 비어있으면 true, 아니면 false를 반환

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <Vector>
3
4  using namespace std;
5
6  int main() {
7      vector<int> v = {10, 20, 30};
8
9      v.insert(v.begin(), 0); // {0, 10, 20, 30}
10     v.push_back(40); // {0, 10, 20, 30, 40}
11
12     cout << v.front() << " " << v.back() << "\n";
13
14     v.erase(v.begin() + 1); // {0, 10, 30, 40}
15     v.pop_back(); // {0, 20, 30}
16
17     cout << v[1] << " " << v.back();
18
19     return 0;
20 }
```

Input: [Copy](#)

Expected Output: [Copy](#)

0 40
20 30

`v.front(), v.back()` | `<vector>` | $O(1)$

- 각각 벡터의 맨 앞, 뒤의 원소를 반환

`v.push_back(val)` | `<vector>` | $O(1)$

- 벡터의 맨 뒤에 원소 추가

`v.pop_back()` | `<vector>` | $O(1)$

- 벡터의 맨 뒤 원소 삭제

`v.insert(iterator, val)` | `<vector>` | $O(n)$

- 특정 위치에 원소 삽입
- 기존에 있던 원소들이 뒤로 한 칸 씩 밀림

`v.erase(iterator)` | `<vector>` | $O(n)$

- 특정 위치의 원소 삭제
- 기존에 있던 원소들이 앞으로 한 칸 씩 당겨짐

4. 기초 자료구조: vector



`vector<int> v = {10, 20, 30, 40, 50};`

10	20	30	40	50
----	----	----	----	----

`v.insert(v.begin(), 0);`

0	10	20	30	40	50
---	----	----	----	----	----

`v.erase(v.begin() + 1);`

0	20	30	40	50
---	----	----	----	----

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      vector<int> v = {1, 3, 4, 5, 2};
9
10     sort(v.begin(), v.end());
11
12     for(int i = 0; i < v.size(); i++) {
13         cout << v[i] << " ";
14     }
15
16     cout << "\n";
17
18     reverse(v.begin(), v.end());
19
20     for(int i = 0; i < v.size(); i++) {
21         cout << v[i] << " ";
22     }
23
24     return 0;
25 }
```

sort(시작 주소, 끝 주소) | `<algorithm>` | $O(n \log n)$

- 벡터의 지정한 범위에 속한 원소들을 오름차순으로 정렬

reverse(시작 주소, 끝 주소) | `<algorithm>` | $O(n)$

- 벡터의 지정한 범위에 속한 원소들의 순서를 뒤집음

Input:

Copy

Expected Output:

Copy

1 2 3 4 5
5 4 3 2 1

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      vector<int> v = {1, 2, 2, 4, 5, 9, 9, 10};
9
10     int cnt2 = count(v.begin(), v.end(), 2);
11
12     int maxVal = *max_element(v.begin(), v.end());
13     int minVal = *min_element(v.begin(), v.end());
14
15     cout << cnt2 << " " << maxVal << " " << minVal;
16
17     return 0;
18 }
```

Input:

Copy

Expected Output:

Copy

2 10 1

count(시작 주소, 끝 주소, val) | <algorithm> | $O(n)$

- 찾는 값이 해당 벡터의 지정한 범위에 몇 개 있는지 반환

max_element(시작 주소, 끝 주소) | <algorithm> | $O(n)$

min_element(시작 주소, 끝 주소) | <algorithm> | $O(n)$

- 각각 해당 벡터의 최댓값과 최솟값을 찾고 해당 원소의 포인터를 반환
- 결과값으로 반복자를 반환하기에 앞에 *를 붙여 해당 값에 접근할 수 있음.

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      vector<int> v = {1, 2, 2, 4, 5, 9, 9, 10};
9      int target = 9;
10
11     auto it = find(v.begin(), v.end(), target);
12
13     if(it != v.end()) {
14         cout << "find " << *it << "\n";
15         cout << "index : " << it - v.begin();
16     }
17     else cout << "can't find " << target;
18
19     return 0;
20 }
```

find(시작 주소, 끝 주소, val) | `<algorithm>` | $O(n)$

- 벡터 안의 특정값을 찾아내는 함수
- 결과값으로 반복자를 반환
- 앞에서부터 순서대로 특정값을 찾음
- 해당 값이 벡터에 2개 이상 포함되어있을 경우 처음에 찾은 값의 반복자를 반환
- 해당 값을 찾지 못했을 경우 `v.end()` 반복자 반환

auto 자료형

- 선언된 변수의 초기값을 보고 컴파일러가 실제 자료형을 추론해주는 자료형
- 해당 예제에서는 auto를 사용하지 않았으면 `vector<int>::iterator`를 사용해야 함

Input:

Copy

Expected Output:

Copy

find 9
index : 5

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <vector>
3  #include <algorithm>
4
5  using namespace std;
6
7  int main() {
8      vector<int> v = {1, 2, 3, 4, 5};
9
10     fill(v.begin() + 1, v.begin() + 3, 6);
11
12     for(int i : v) {
13         cout << i << " ";
14     }
15
16     return 0;
17 }
```

Input: Copy

Expected Output: Copy

1 6 6 4 5

fill(시작 주소, 끝 주소, val) <algorithm> | $O(n)$

- 벡터의 지정한 범위를 특정 값으로 초기화 함

범위 기반 for문 : for(자료형 변수명 : 컨테이너명)

- 처음부터 끝까지 모든 원소를 순회할 때 사용하는 방법
- 인덱스를 쓰지 않아서 범위 초과, 미만 오류를 원천 차단할 수 있음
- 컨테이너의 끝에 도달하면 알아서 반복을 멈춤

4. 기초 자료구조: vector

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4
5  int main() {
6      vector<vector<int>> v1;
7
8      vector<vector<int>> v2(3, vector<int>(5));
9
10     vector<vector<int>> v3(2, vector<int>(4, 1));
11
12     vector<vector<int>> v4 = {
13         {1, 2, 3},
14         {4, 5},
15         {6, 7, 8, 9}
16     };
17
18     cout << v4[2][2];
19
20     return 0;
21 }
```

2차원 벡터 개요

- 표 형태의 데이터를 다루기 위한 구조
- 벡터를 원소로 가지는 벡터
- 행과 열로 이루어져 있으며, 1차원 벡터와 선언, 초기화, 인덱싱은 동일함
- $v[i][j]$ 에서 행과 열이 헷갈리면 건물의 '층 j호'라고 생각하면 쉬움

2차원 벡터 선언

- v1 : 2차원 벡터를 선언한 코드
- v2 : 3행 5열 벡터를 만들고 0으로 초기화하는 코드
- v3 : 2행 4열 벡터를 만들고 1로 초기화 하는 코드
- v4 : 벡터에 직접 원소를 부여하는 코드

Input:

Copy

Expected Output:

Copy

8

4. 기초 자료구조: vector



특징	일반 배열 (int arr[n])	벡터 (std::vector<int> v)
크기 결정	정적	동적
메모리 위치	스택(Stack)	힙(Heap)
공간 추가 및 삭제	불가능	v.push_back(), v.push_pop() 등 으로 자유롭게 추가 및 삭제
편의성 함수	거의 없음	풍부함
속도	최상(직접 참조)	우수(간접 참조)

4. 기초 자료구조: vector



실습

- [10807 개수 세기](#)
- [10818 최소, 최대](#)
- [2738 행렬 덧셈](#)

4. 기초 자료구조: pair

```
1  #include <iostream>
2  #include <utility>
3  #include <string>
4
5  using namespace std;
6
7  int main() {
8      pair<int, string> p1;
9      pair<int, string> p2 = {10, "apple"};
10
11      p1.first = 20;
12      p1.second = "banana";
13
14      cout << p1.first << " " << p1.second << "\n";
15      cout << p2.first << " " << p2.second;
16
17      return 0;
18 }
```

페어(pair) | `<utility>`

- 두 개의 같거나 다른 자료형의 데이터를 하나의 쌍으로 묶는 자료구조
- 좌표(x, y), 성적(이름, 점수) 등 서로 연관된 두 데이터를 한 번에 관리할 때 사용

선언과 초기화

- p1 : int와 string를 원소로 가지는 페어를 선언
- p2 : int와 string을 원소로 가지는 페어를 선언하고 원소를 부여함

p.first, p.second) | `<utility>` | $O(1)$

- 페어의 첫 번째 데이터는 .first, 두 번째 데이터는 .second로 접근

Input:

Copy

Expected Output:

Copy

20 banana
10 apple

4. 기초 자료구조: pair

```
1  #include <iostream>
2  #include <vector>
3  #include <utility>
4
5  using namespace std;
6
7  int main() {
8      vector<pair<int, int>> points;
9
10     points.push_back({3, 4});
11     points.push_back({1, 1});
12     points.push_back({1, 5});
13     points.push_back({2, 2});
14
15     for (int i = 0; i < points.size(); i++) {
16         cout << points[i].first << ", ";
17         cout << points[i].second << "\n";
18     }
19
20     return 0;
21 }
```

벡터와의 사용

- 여러 개의 '데이터 쌍'을 하나의 벡터에서 관리하는 자료구조
- 데이터를 추가하거나 정렬할 때 값이 섞이는 일을 방지할 수 있음

Input:	Copy
Expected Output:	Copy
3, 4	
1, 1	
1, 5	
2, 2	

4. 기초 자료구조: pair



실습

- [2711 오타맨 고창영](#)

5. 잡다하지만 중요한 내용

1. 입출력 속도 비교

- 기존 C의 표준 입출력(`printf`, `scanf`)과 C++의 `cin`, `cout`는 성능 차이가 있을까?
 - C의 표준 입출력이 압도적으로 빠름
 - `cin`과 `cout`는 기본적으로 C의 표준 입출력과 혼합해서 사용해도 순서가 꼬이지 않게 동기화 되어있음
 - 이 동기화를 유지하는 부분이 입출력을 느리게 만드는 원인
 - 이를 해결하기 위해 동기화를 끊을 필요가 있음
-
- `ios_base::sync_with_stdio(false);`
 - `cin.tie(NULL);`

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      ios_base::sync_with_stdio(false);
7      cin.tie(NULL);
8
9      int a, b;
10
11     cin >> a >> b;
12
13     cout << a + b;
14
15     return 0;
16 }
```

5. 잡다하지만 중요한 내용



실습

- 15552 빠른 A + B
- 10818 최소, 최대

5. 잡다하지만 중요한 내용

2. 캐스팅(Casting)

- 변수의 자료형을 다른 자료형으로 강제로 변환하는 행위
- 서로 다른 자료형끼리 연산하거나, 함수에서 요구하는 자료형이 현재 자료형과 다를 때 사용
- 묵시적 변환과 명시적 변환으로 나누어짐

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int a = 5;
7      double b = 11;
8      char c = 'A';
9
10     cout << b / a << "\n";
11     cout << c + a;
12
13     return 0;
14 }
```

Expected Output:

Copy

Received Output:

Set Copy

2.2
70

```
1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int a = 5;
7      double b = 11;
8      char c = 'A';
9
10     cout << (int)b / a << "\n";
11     cout << (char)c + a;
12
13     return 0;
14 }
```

Expected Output:

Copy

Received Output:

Set Copy

2
70

5. 잡다하지만 중요한 내용



2. 캐스팅(Casting)

- 단, string 자료형의 경우 함수를 사용해야 함
- int -> string의 경우 to_string()
- string -> int의 경우 stoi()

```
1  #include <iostream>
2  #include <string>
3
4  using namespace std;
5
6  int main() {
7      string s1 = "12345";
8      string s2;
9      int n1;
10     int n2 = 67890;
11
12     n1 = stoi(s1);
13     s2 = to_string(n2);
14
15     cout << n1 + n2 << "\n" << s1 + s2;
16
17     return 0;
18 }
```

Input:

Copy

Expected Output:

Copy

80235
1234567890

5. 잡다하지만 중요한 내용



실습

- [11654 아스키 코드](#)